**Administrative**

Team Name

Path Planners!

Team Members

Vignesh Saravanan (vsaravanan1)
Devon Giuliani (ThePillowMan)
Austin Taylor (austinktaylors)

Github Repo:

https://github.com/vsaravanan1/PathPlanningDSA/tree/main

Video Demo

https://www.youtube.com/watch?v=3zs9eRqwKiQ

**Extended and Refined Proposal**

Problem

Do search-based methods (such as A*) or sampling-based methods (such as RRT) work better for rover path planning on unexplored lunar terrain?

Motivation

Any rover deployed in space has strict limits in terms of both energy and compute, and the supervising engineers back on Earth will have limited direct control over the rover. As such, it is important that the rover is able to navigate an unknown environment efficiently and complete its tasks so that the mission is successful. As such, we need an efficient path planning algorithm.

Features Implemented

There are 12 occupancy grids that correspond to different regions of lunar terrain. Both algorithms were run on all 12 occupancy grids with the same randomly chosen start and goal points. Comparisons were performed between both algorithms in terms of path efficiency (closest to straight-line distance) and time taken to compute. There is an overall comparison script that compares average values for these metrics over all occupancy grids, and a command-line-interface that does comparison for a single occupancy grid at a time.

Description of Data

Occupancy grids for various maps on the moon: MoonPlanBench.

Tools/Languages/APIs/Libraries Used

Python, numpy, scipy, built-in collections such as the heap, GitHub.

Algorithms Implemented

Three algorithms were implemented for this project: the RRT, RRT*, and A* algorithms. The RRT and RRT* algorithm was implemented as our chosen sampling based algorithm. The RRT works by randomly generating points on a map. The closest point to the randomly generated point then gains a new node a short distance from it in the direction of the random point. This is continued until the goal is reached. The RRT* algorithm works mostly the same, but additionally shortens paths when possible based on minimum total cost from start node. The A* algorithm was implemented as our chosen search based algorithm. This works by using a formula to give neighboring nodes a heuristic cost-to-go representative of their distance from the end point. The one with the smallest value is chosen, and the process then continues (remembering the values of any previous neighbours already calculated).

Additional Data Structures/Algorithms Used

No additional data structures were used. However, one additional algorithm was implemented: a double-BFS flood fill. The flood fill algorithm was used to find a good starting and ending point at random (one which is distant, can be reached, and has to traverse around obstacles).

Distribution of Responsibility and Roles

Vignesh: Focus on data structure/organization and implementation of sampling based methods (RRT implementation)

Devon: Focus on data structure/organization and implementation of search based methods (A* implementation)

Austin: Data reading, data structure comparison, and interface development

**Analysis**

Changes Made After the Proposal

There were no significant changes made after the proposal. The only thing changed was that the code generates a matplotlib plot visually showing the path chosen.

Big O Time Complexity

The Worst Case time complexity is explained in depth inside its own section following this.

**Worst-case Time Complexity Analysis of Functions Implemented:**

Helper Data Structure: Dynamic KDTree (wrapper around KDTree from scipy.spatial)
Parameters: n - number of nodes in tree

- Insert: O(nlog(n)), since the tree is rebuilt every 50 nodes added, and insertion of a single node into a tree is O(log(n)) (n nodes are inserted when the tree is rebuilt). It is also O(1) amortized, since for 49 out of 50 insertions, the node is just appended to the pending buffer.
- Query: O(log(n)), since searching in a regular KDTree is O(log(n)), and the tree contains a number of nodes on the order of n. The pending buffer contains only up to 50 nodes, and so searching within the buffer can be done in constant time complexity (or O(1)).

RRT Function time complexity:
Parameters: n - max_tree_size (limits number of nodes)
expand_tree**:**

- Involves a call to search_tree.query (O(log(n)), since tree can contain at most n nodes
- Gets local path using get_local_path (between nearest node currently on tree and another candidate): distance of this path is limited to a constant by the step size parameter (O(1) for fixed distance, since local path planner just attempts a straight line)
- Inserts candidate into search tree (O(log(n) operation in the worst case)
- If the goal is discovered, it is retraced back to the start node, which has a time complexity of O(n), since at most n nodes can be traversed along the route to the start node. At each stage, a dictionary is used to retrieve the parent of a given node, and so finding the parent is an O(1) operation.
- Thus, the worst case time complexity of expand_tree is O(n + 1 + 2log(n)) = O(n)

**run:**

- With standard RRT, we stop the moment we discover the goal, which means that the path retracing occurs only on a single iteration.
- Thus, we call **expand_tree** with retracing only a maximum of one time. The number of nodes is limited by n. Hence, the worst case time complexity is O(n * log(n) + n) = O(nlog(n)).

RRT* Function Time Complexity:
Parameters:
n - max_tree_size (limits number of nodes)
k - maximum number of neighbors within a given radius
expand_tree**:**

- The main new component to “expand tree” is the neighbor modification that occurs, which involves calling query_ball_point on the Dynamic KDTree (an O(log(n) operation,

where n is the number of nodes in the tree) and iterating through the list of neighbors and performing O(1) operations for each neighbor. This results in an overall worst-case time complexity of O(k + log(n)) with no path-retracing and O(k + n) with path retracing.

Run**:**

- There are at most 10 iterations of **expand_tree**, and the algorithm stops only when the tree is full, not necessarily when a goal is found. In the worst case, we would be constantly aiming toward the goal point and it would be reachable in each step, which means we would be doing the path retracing for every iteration, which would cause the overall time complexity to become O((k+ n)(n)) = O(n^2 + kn).

A* Function Time Complexity:
Parameters:
n - max_graph_size (row count multiplied by column count)
Run**:**

- There are three main functions inside the A* algorithm programmed, the while loop code, the if statement checking whether the target has been met, the for loop, and the final conditional of pushing a neighbour. Since the while loop occurs for however nodes are in the openHeap, theoretically, the worst case time complexity would be O(n), where every node is added to the heap since the target is not inside. The check of the target and the final code adding up the path would be O(n) since the final returned value goes through every value in the path. In the instance the path travels the entire graph, there would be n iterations. The for loop would be O(1) since it only runs for every neighbour of the current node, and the number of neighbours is constant. Lastly, the final loop pushing a neighbor has two separate worst case time complexities (one for if the neighbour is already in the open heap and one for if it is): O(n) and O(logn). O(n) would occur since every time a neighbour is already in the open heap, heapify occurs. Heapify - itself - has a time complexity of O(n). Alternatively O(logn) occurs when it's not in an open heap because the heap is a complete tree (and traversal of a complete tree is O(logn)). Overall, this program runs at O(n^2) in the worst case where either there is no node (meaning every node is checked and heapify occurs multiple times) or the path extends the entire graph.

**Reflection**

Overall Experience

Vignesh: I had a great experience working with Devon and Austin. We made a plan at the start and everyone fulfilled their tasks.

Devon: I had a great experience working with Vignesh and Austin and really learned a lot about team dynamics. Since everyone was so willing to help out on the project and a good structure/plan was made, this project went very smoothly.

Austin: I had a good time working with Vignesh and Devon and am happy with what we were able to accomplish as a team

Challenges

Some major challenges faced in this project were communication and knowledge. Communication was difficult inside our group since we were all available at different times and days, with Vignesh living in a different time zone altogether. Another big hurdle was knowledge, since we had a very small basis of knowledge regarding our chosen sampling and search methods, requiring further research into how to implement said algorithms. Additionally, some of us had troubles regarding the Python language, needing to refresh their knowledge on how to best utilize it. Lastly, there was some confusion regarding what exactly we were tackling at the very start of the project.

Changes Made in Retrospect

In retrospect, our group would likely not change much. Other than potentially clarifying what exactly moon path planning was, most of the project went rather smoothly. Some procrastination did occur which would - again - likely not have occurred in retrospect, but other than that and clarifying what is to be done pertaining to path planning, no changes would be made in retrospect.

Knowledge Learned

Vignesh: I learned the benefits and drawbacks of search and sampling-based methods by implementing RRT and RRT* myself and also reading through Devon's implementation of A*. I also learned more about utilizing numpy for vectorization and fast execution of algorithms.

Devon: Through this project, I learned how to properly implement and develop an A* algorithm, as well as how the RRT, RRT*, and A* algorithm work. Additionally, I learned how teams best work together.

Austin: I learned about the three path planning algorithms and also about how to architect a good command line interface to test all kinds of algorithms and report results.

**References**

Planetary Terrain Datasets and Benchmarks for Rover Path Planning
A* Pathfinding (E01: algorithm explanation)
MoonPlanBench